

Reinforcement Learning for Sim-to-Sim Robot Transfer

Leonardo Passafiume (358616)

Lucio Baiocchi
Thomas Kelly

Maurizio Lanzoni Raiteri

Abstract

This report studies reinforcement learning for continuous control and sim-to-real transfer in the simplified sim-to-sim setting proposed by the project. First, we implemented REINFORCE, REINFORCE with a constant baseline, and an actor-critic policy-gradient variant from scratch on Hopper-v4. Then, we moved to the PandaPush-v3 manipulation task and trained Stable-Baselines3 PPO and SAC agents under source and target dynamics, where the target object is heavier than the source object. The final implementation includes an evaluation pipeline for fixed and randomized object mass and friction, a reward-shaping wrapper, Uniform Domain Randomization (UDR) over mass and friction, and a mass-only Adaptive Domain Randomization (ADR) wrapper. On the currently available deterministic 50-episode SAC evaluations, the source-trained SAC policy reaches 98% success on source and 96% success on target at the nominal target mass, while the target-trained oracle reaches 100% on target. Under a harder randomized evaluation with mass sampled in [5, 10] kg and randomized friction, the source-trained non-randomized SAC policy drops to 76% success, whereas the UDR policy trained with mass and friction randomization reaches 90%. PPO, ADR, multi-seed statistics, and the final Part 1 numerical table are not yet complete and are explicitly marked as TODO where needed.

1. Introduction

Reinforcement learning (RL) formulates control as sequential decision making, where an agent learns a policy by interacting with an environment and maximizing expected return [10]. In robotics this setting is attractive because it can optimize behaviors that are difficult to hand-design, but it is also difficult because interactions are expensive, exploration can be unsafe, and the learned policy can exploit simulator-specific details [5, 6]. A common practical solution is to train in simulation and transfer the policy to the real robot. The central difficulty is the reality gap: the simulator dynamics are never exactly equal to the deployment dynamics [4].

This project follows the course sim-to-real abstraction in a controlled sim-to-sim benchmark. In Part 1, we implement basic policy-gradient algorithms on Hopper-v4 to study the variance and stability issues behind direct policy optimization. In Part 2, we use PandaPush-v3, where the source domain has a light object and the target domain has a heavier object. The target domain plays the role of the real world: it is available for evaluation, but training directly on it is treated as an oracle upper bound rather than as the deployable solution.

The second part uses modern actor-critic algorithms from Stable-Baselines3. PPO constrains policy updates through a clipped surrogate objective [9], while SAC is an off-policy maximum-entropy method that learns stochastic policies and reuses data through a replay buffer [3]. To reduce the transfer gap, we implement domain randomization, which trains over a distribution of simulator parameters instead of a single nominal model [7, 8, 11]. The implemented UDR wrapper samples parameters from fixed ranges, while the ADR wrapper adapts the mass range based on recent success, following the curriculum idea behind automatic domain randomization [1].

The report focuses on what was implemented in this repository: the algorithmic choices, the wrappers, the training and evaluation protocol, and the currently available results. Missing experiments are not filled with invented values; they are marked explicitly as TODO.

2. Methodology

2.1. Part 1: Policy Gradients on Hopper

The first part uses Hopper-v4, a continuous-control MuJoCo task. The action is continuous and the policy is stochastic. In `part1/agent.py`, the policy network is a two-hidden-layer MLP with 64 units per layer and $\tanh$ activations. The actor outputs the mean of a diagonal Gaussian distribution; the standard deviation is a learned parameter passed through `softplus`. All Part 1 experiments use Adam with learning rate 10^{-3} and discount factor $\gamma = 0.99$.

REINFORCE. For an episode trajectory, the implementation computes Monte-Carlo returns

$$G_t = \sum_{k=t}^T \gamma^{k-t} r_k, \quad (1)$$

and minimizes the negative policy-gradient objective

$$\mathcal{L}_{PG} = - \sum_t (G_t - b) \log \pi_\theta(a_t | s_t), \quad (2)$$

where b is either zero or a constant baseline. The implemented command-line interface allows changing the constant baseline with `--baseline`. The available plot file for the baseline run uses $b = 20$.

Actor-Critic Variant. The actor-critic implementation uses the same actor network plus a critic head that predicts $V_\phi(s)$. The critic is trained by regression to Monte-Carlo returns, and the actor uses the estimated advantage

$$A_t = G_t - V_\phi(s_t). \quad (3)$$

Thus, the implementation is an actor-critic policy-gradient variant with a learned value-function baseline, not a purely one-step TD actor-critic. Training and evaluation curves are saved under `part1/plots/`. Final multi-seed numerical summaries are still missing.

2.2. Part 2: PandaPush Pipeline

The second part uses the local `panda-gym` copy. In `part2/panda-gym/panda-gym/envs/tasks/push.py`, the source object mass is 1.0 kg and the target object mass is 5.0 kg. The goal position is fixed at the center of the table, while the object initial position is randomized in the xy plane. The main training scripts use the dense task reward and an episode horizon of 500 steps.

Stable-Baselines3 Training. The main unified script is `part2/train_sb3.py`. It supports PPO and SAC with `MultiInputPolicy`. The PPO configuration exposes learning rate, discount, number of steps, clipping range, entropy coefficient, GAE lambda, and the number of parallel environments. The SAC configuration exposes learning rate, discount, batch size, replay-buffer size, learning starts, target-network update coefficient, train frequency, gradient steps, and optional Hindsight Experience Replay (HER) [2]. HER is implemented through the Stable-Baselines3 `HerReplayBuffer` with `n_sampled_goal=4` and `goal_selection_strategy="future"`.

The representative SAC runs currently saved in `models/` were trained for 500k steps with seed 42. The best source SAC run uses learning rate 10^{-3} , batch size 2048, replay buffer size 10^6 , learning starts 100k, $\tau =$

0.05, and $\gamma = 0.95$, according to `part2/commands.md`. **[TODO: Confirm in the final report whether each saved SAC checkpoint used HER; the model directory names do not encode this if a custom run name was used.]**

Reward Shaping. The wrapper `part2/wrappers.py` changes only the scalar training reward, not the dynamics. The shaped reward is

$$r_{\text{train}} = r_{\text{env}} - p + B \mathbf{1}[d(o, g) < \epsilon], \quad (4)$$

where p is a per-step penalty, B is a close-goal bonus, and $d(o, g)$ is the object-goal distance. In the unified script the values are $p = 10^{-2}$, $B = 1.0$, and $\epsilon = 0.05$. Evaluation uses the environment reward directly, so success rate is not computed from the shaped reward.

Domain Randomization. The wrapper `part2/rand_wrapper.py` implements three modes. In `none`, the nominal environment dynamics are kept. In `udr`, the wrapper samples a new mass and friction values at every reset. The current UDR training ranges are:

$$m \sim \mathcal{U}(1, 5), \quad \mu_{\text{obj}}, \mu_{\text{table}} \sim \mathcal{U}(0.5, 1.2), \quad (5)$$

and object spinning friction is sampled from $\mathcal{U}(0, 0.005)$. The mass is applied through PyBullet `changeDynamics`, while lateral and spinning friction are applied through the local PyBullet helper methods. In `adr`, the wrapper currently adapts only the mass range. It starts from the nominal mass and stores a window of 20 episode successes. If the mean success rate is at least 0.8, the mass range expands by 0.25 kg on each side within the global limits; if it is at most 0.2, the range contracts toward the nominal mass. Friction remains nominal in ADR.

Evaluation Protocol. The evaluation script `part2/eval_sb3.py` supports SAC and PPO checkpoints. It reports mean return, standard deviation, min/max return, final success rate from `info["is_success"]`, and parameter-conditioned summaries. Standard fixed-domain evaluations use 50 deterministic episodes and seed 42. The harder stress evaluation samples mass uniformly in $[5, 10]$ kg, object and table lateral friction in $[0.2, 1.5]$, and object spinning friction in $[0, 0.01]$. Center-of-mass randomization was discussed as a useful extension, but it is not part of the current repository state used for this draft. **[TODO: Add center-of-mass randomization results only after the implementation and evaluation are present in the repo.]**

3. Experimental Results

3.1. Part 1: Hopper

The Part 1 implementation produced learning-curve plots for REINFORCE without baseline, REINFORCE with a constant baseline of 20, and the actor-critic variant. The plots show that all three runs can eventually reach high returns, but the learning curves are noisy and single-seed only. The constant baseline improves the early REINFORCE curve relative to the no-baseline run, but it does not remove instability. The actor-critic run uses a learned value baseline and reaches high return near the end of training, although it also exhibits long low-return phases.

Method	Seed	Episodes	Final metric
REINFORCE	0	3000	[TODO: eval]
REINFORCE, $b = 20$	0	3000	[TODO: eval]
Actor-critic	0	3000	[TODO: eval]

Table 1. Part 1 runs currently available as plots in `part1/plots/`. The final report should fill this table with deterministic evaluation means and standard deviations, preferably over at least three seeds.

3.2. Part 2: Training Diagnostics

SAC was selected as the main Part 2 algorithm because the available logs show faster and more reliable success than PPO. The clearest SAC runs reach high training success after 500k steps. PPO logs are more variable: some runs remain near 0.15–0.19 success even after many updates, while one run reaches 0.87 around 2M steps. This is consistent with the expected advantage of off-policy SAC on goal-conditioned manipulation, especially when replay and optional HER are available.

Run	Step	Train SR
SAC none source	499700	0.97
SAC none target	499933	1.00
SAC UDR source	499988	0.99
SAC UDR + friction	499957	0.96
Best logged PPO run	2015232	0.87

Table 2. Final TensorBoard training success rates from available logs. These are training diagnostics, not a substitute for the deterministic evaluation table.

3.3. Nominal Sim-to-Sim Evaluation

Table 3 reports deterministic 50-episode evaluations generated from the saved SAC checkpoints. At the nominal target mass of 5 kg, the source-trained SAC policy already transfers well: the gap between source→source and

source→target is small in success rate. The target-trained model is still the oracle upper bound.

3.4. Harder Mass and Friction Evaluation

Because nominal mass transfer was already strong, the evaluation was extended to randomize parameters at test time. Table 4 uses mass sampled uniformly in $[5, 10]$ kg, object/table lateral friction sampled in $[0.2, 1.5]$, and object spinning friction sampled in $[0, 0.01]$. Under this harder protocol, the non-randomized source policy drops from 96% to 76% success on target. The UDR policy trained with mass and friction randomization reaches 90%, suggesting that UDR is more useful when the evaluation distribution is wider than the original source-target mass shift.

Model	Mean return	Std	Success
SAC none source	-10.990	17.411	76%
SAC UDR + friction	-5.004	14.170	90%

Table 4. Hard target evaluation over 50 deterministic episodes, seed 42. The sampled masses had min 5.219 kg, mean 7.676 kg, and max 9.878 kg.

3.5. Missing Final Experiments

The following results are still required before the report can be considered final: PPO deterministic evaluation with valid normalization statistics, ADR source→source and source→target evaluation, multi-seed means and standard deviations, and the final Part 1 numeric table. **[TODO: Run the mandatory final grid and replace the TODO cells before submission.]**

4. Discussion

4.1. Result Analysis

The most important observation is that the nominal source→target transfer is easier than expected from mass alone. A source-trained SAC policy reaches 96% success on the 5 kg target object, only slightly below its 98% source success. This suggests that the current `PandaPush-v3` setup is not very sensitive to the fixed 1 kg to 5 kg mass shift. The task has a fixed goal, a short object-goal distance distribution, dense rewards during training, and a powerful continuous-control algorithm. These factors can hide part of the intended reality gap.

The harder randomized evaluation is therefore more informative. When target mass is sampled in $[5, 10]$ kg and friction is randomized, the non-randomized SAC policy falls to 76% success. This indicates that friction and heavier masses expose failures that are not visible in the nominal target evaluation. The UDR policy trained with mass and friction randomization performs worse than the nominal

Model	Evaluation	Mean return	Std return	Success	Mass
SAC none source	source	-1.536	7.574	98%	1 kg
SAC none source	target	-2.316	9.578	96%	5 kg
SAC none target	target	-0.429	0.274	100%	5 kg
SAC UDR + friction source	source	-9.761	46.666	94%	1 kg
SAC UDR + friction source	target	-6.509	26.756	94%	5 kg

Table 3. Available 50-episode deterministic SAC evaluations with fixed nominal mass and fixed nominal friction, seed 42. The UDR checkpoint is the saved `SAC_uds_source_500k_friction` model.

SAC policy on the easy fixed evaluation, but better on the hard evaluation. This is the expected robustness trade-off: UDR may sacrifice performance on the nominal dynamics in exchange for a policy that is less specialized.

SAC appears more suitable than PPO for this repository state. The available logs show high SAC training success after 500k steps, while PPO runs are less consistent and require careful treatment of vector normalization. This is coherent with the algorithms: SAC is off-policy and can reuse data through its replay buffer, whereas PPO is on-policy and discards rollouts after each update. For goal-conditioned pushing, replay-based learning is especially useful because informative transitions can be reused many times.

4.2. Reward Shaping

The reward-shaping wrapper affects training but not the evaluation success metric. The time penalty encourages short solutions, and the close-goal bonus increases the learning signal once the object reaches the success threshold. For SAC, this can increase success rate indirectly by making successful behavior easier to learn early in training. However, it also makes training return less comparable to raw evaluation return. This is why the evaluation script reports both return and success rate on the unwrapped environment.

4.3. Methodological Critique

The main limitation is that the current result set is not yet statistically complete. Most numbers are single-seed evaluations with 50 episodes. They are useful for debugging and for identifying trends, but the final report should include mean and standard deviation across seeds where possible.

The domain-randomization implementation is also intentionally simple. UDR uses hand-selected uniform ranges, so its behavior depends strongly on those ranges. If the training range matches the evaluation range too closely, the comparison becomes too generous. The current setup avoids the most direct form of this issue by using conservative training friction ranges and a harder evaluation range, but a final study should report the exact train and test distributions clearly.

ADR is currently mass-only and uses a simple success-rate heuristic. It does not adapt friction, center of mass,

action noise, observation noise, or target sampling. Therefore, any ADR result should be interpreted as a curriculum over object mass rather than as a full automatic domain-randomization method. In addition, the source-domain success rate may be a weak curriculum signal for transfer: a policy can be successful in the randomized training domain while still failing on the target distribution.

Finally, PPO evaluation needs care because some saved PPO models were trained behind `VecNormalize`. A valid deterministic PPO evaluation must load the corresponding normalization statistics; otherwise the policy sees observations on a scale different from training. This is marked as TODO instead of reporting unreliable numbers.

5. Conclusion

This project implemented the full path from basic policy-gradient methods on Hopper to sim-to-sim transfer experiments on PandaPush. In the current repository state, SAC is the strongest and most reliable Part 2 algorithm. At the nominal 5 kg target mass, the source-trained SAC policy already transfers well, reaching 96% success over 50 deterministic episodes, while the target-trained oracle reaches 100%. A harder mass-and-friction evaluation reveals a clearer reality gap: the non-randomized source policy drops to 76% success, while the UDR policy trained with mass and friction randomization reaches 90%. The final conclusions still require PPO, ADR, multi-seed, and Part 1 numerical evaluations to be completed.

References

- [1] Ilge Akkaya, Marcin Andrychowicz, Maciek Chociej, Mateusz Litwin, Bob McGrew, Arthur Petron, Alex Paino, Matthias Plappert, Glenn Powell, Raphael Ribas, Jonas Schneider, Nikolas Tezak, Jerry Tworek, Peter Welinder, Lilian Weng, Qiming Yuan, Wojciech Zaremba, and Lei Zhang. Solving rubik’s cube with a robot hand. *arXiv preprint arXiv:1910.07113*, 2019. 1
- [2] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. In *Advances in Neural Information Processing Systems*, 2017. 2
- [3] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep

- reinforcement learning with a stochastic actor. In *International Conference on Machine Learning*, pages 1861–1870, 2018. [1](#)
- [4] Sebastian Hofer, Kostas Bekris, Ankur Handa, Juan Camilo Gamboa, Melissa Mozifian, Florian Golemo, et al. Perspectives on sim2real transfer for robotics: A summary of the r:ss 2020 workshop. *arXiv preprint arXiv:2012.03806*, 2020. [1](#)
 - [5] Jens Kober, J. Andrew Bagnell, and Jan Peters. Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research*, 32(11):1238–1274, 2013. [1](#)
 - [6] Petar Kormushev, Sylvain Calinon, and Darwin G. Caldwell. Reinforcement learning in robotics: Applications and real-world challenges. *Robotics*, 2(3):122–148, 2013. [1](#)
 - [7] Fabio Muratore, Tim Gruner, Florian Wiese, Boris Belousov, Michael Gienger, and Jan Peters. Robot learning from randomized simulations: A review. *Frontiers in Robotics and AI*, 9, 2022. [1](#)
 - [8] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *IEEE International Conference on Robotics and Automation*, pages 3803–3810, 2018. [1](#)
 - [9] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. [1](#)
 - [10] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, second edition, 2018. [1](#)
 - [11] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. *arXiv preprint arXiv:1703.06907*, 2017. [1](#)